1.AVLTreeProgram.java

```
import java.io.*;
import java.util.*;

// Class for AVL Tree operations
class AVLTreeProgram {

  // Node structure of AVL Tree
  class Node {
    int key;      // Value stored in node
    int height;    // Height of node
    Node left, right; // Left and right child pointers

    Node(int d) {
      key = d;
      height = 1; // New node initially added at leaf
    }
  }

  Node root; // Root node of AVL tree

  // Function to get height of node
  int height(Node N) {
    if (N == null)
      return 0;
    return N.height;
  }
```

```
// Function to return maximum of two values
int max(int a, int b) {
   return (a > b) ? a : b;
}

// Right rotation (used in LL case)
Node rightRotate(Node y) {

   Node x = y.left;
   Node T2 = x.right;

   // Perform rotation
   x.right = y;
   y.left = T2;

   // Update heights
   y.height = max(height(y.left), height(y.right)) + 1;
   x.height = max(height(x.left), height(x.right)) + 1;

   return x; // New root after rotation
}

// Left rotation (used in RR case)
Node leftRotate(Node x) {

   Node y = x.right;
   Node T2 = y.left;
```

```
    // Perform rotation
    y.left = x;
    x.right = T2;

    // Update heights
    x.height = max(height(x.left), height(x.right)) + 1;
    y.height = max(height(y.left), height(y.right)) + 1;

    return y; // New root after rotation
}

// Function to get balance factor
int getBalance(Node N) {

    if (N == null)
        return 0;

    return height(N.left) - height(N.right);
}

// Function to insert node in AVL tree
Node insert(Node node, int key) {

    // Step 1: Perform normal BST insertion
    if (node == null)
        return new Node(key);

    if (key < node.key)
```

```
    node.left = insert(node.left, key);

else if (key > node.key)
    node.right = insert(node.right, key);

else
    return node; // Duplicate keys not allowed

// Step 2: Update height
node.height = 1 + max(height(node.left), height(node.right));

// Step 3: Get balance factor
int balance = getBalance(node);

// Step 4: Apply rotations if unbalanced

// LL case
if (balance > 1 && key < node.left.key)
    return rightRotate(node);

// RR case
if (balance < -1 && key > node.right.key)
    return leftRotate(node);

// LR case
if (balance > 1 && key > node.left.key) {
    node.left = leftRotate(node.left);
    return rightRotate(node);
```

```
    }

    // RL case
    if (balance < -1 && key < node.right.key) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }

    return node;
}

// Function to find smallest node
Node minValueNode(Node node) {

    Node current = node;

    while (current.left != null)
        current = current.left;

    return current;
}

// Function to delete node from AVL tree
Node deleteNode(Node root, int key) {

    // Step 1: Perform BST delete
    if (root == null)
        return root;
```

```
if (key < root.key)
   root.left = deleteNode(root.left, key);

else if (key > root.key)
   root.right = deleteNode(root.right, key);

else {

   // Node with one child or no child
   if ((root.left == null) || (root.right == null)) {

      Node temp;

      if (root.left != null)
         temp = root.left;
      else
         temp = root.right;

      // No child case
      if (temp == null)
         root = null;

      else
         root = temp;
   }

   else {
```

```
        // Node with two children
        Node temp = minValueNode(root.right);

        root.key = temp.key;

        root.right = deleteNode(root.right, temp.key);
    }
}

if (root == null)
    return root;

// Step 2: Update height
root.height = max(height(root.left), height(root.right)) + 1;

// Step 3: Check balance factor
int balance = getBalance(root);

// Step 4: Apply rotations if needed

// LL case
if (balance > 1 && getBalance(root.left) >= 0)
    return rightRotate(root);

// LR case
if (balance > 1 && getBalance(root.left) < 0) {
    root.left = leftRotate(root.left);
```

```
        return rightRotate(root);
    }

    // RR case
    if (balance < -1 && getBalance(root.right) <= 0)
        return leftRotate(root);

    // RL case
    if (balance < -1 && getBalance(root.right) > 0) {
        root.right = rightRotate(root.right);
        return leftRotate(root);
    }

    return root;
}

// Function to print inorder traversal on console
void display(Node root) {

    if (root != null) {
        display(root.left);
        System.out.print(root.key + " ");
        display(root.right);
    }
}

// Function to write inorder traversal to file
void inorder(Node root, BufferedWriter bw) throws IOException {
```

```
    if (root != null) {
        inorder(root.left, bw);
        bw.write(root.key + " ");
        inorder(root.right, bw);
    }
}

// Main function
public static void main(String[] args) throws Exception {

    AVLTreeProgram tree = new AVLTreeProgram();

    Scanner sc = new Scanner(System.in);

    // Read input from file
    BufferedReader br = new BufferedReader(new FileReader("input.txt"));

    String line = br.readLine();

    String[] values = line.split(" ");

    // Insert values from file into AVL tree
    for (String val : values)
        tree.root = tree.insert(tree.root, Integer.parseInt(val));

    System.out.println("AVL Tree constructed from file successfully ✅ ");
```

```
        // Insert new element
        System.out.print("Enter element to insert: ");
        int insertVal = sc.nextInt();
        tree.root = tree.insert(tree.root, insertVal);

        // Delete element
        System.out.print("Enter element to delete: ");
        int deleteVal = sc.nextInt();
        tree.root = tree.deleteNode(tree.root, deleteVal);

        // Display inorder traversal
        System.out.print("Inorder traversal: ");
        tree.display(tree.root);

        // Write inorder traversal into output file
        BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));

        tree.inorder(tree.root, bw);

        bw.close();

        System.out.println("\nInorder written to output.txt successfully ✅ ");
    }
}
```

3.Min Heap Delete Root

```
import java.util.Scanner;  // Import Scanner class for user input

// Class definition for Min Heap operations
class MinHeapDeleteRoot {

  int heap[];    // Array to store heap elements
  int size;      // Current number of elements in heap
  int capacity;  // Maximum capacity of heap

  // Constructor to initialize heap
  MinHeapDeleteRoot(int capacity) {
    this.capacity = capacity;     // Set heap capacity
    heap = new int[capacity];     // Create heap array
    size = 0;                     // Initially heap is empty
  }

  // Function to get parent index of a node
  int parent(int i) {
    return (i - 1) / 2;
  }

  // Function to get left child index
  int left(int i) {
    return 2 * i + 1;
  }

  // Function to get right child index
```

```
int right(int i) {
   return 2 * i + 2;
}

// Function to insert element into heap
void insert(int key) {

   // Check if heap is full
   if (size == capacity) {
      System.out.println("Heap overflow!");
      return;
   }

   size++;              // Increase heap size
   int i = size - 1;       // Insert element at last index
   heap[i] = key;

   // Move element upward until heap property is satisfied
   while (i != 0 && heap[parent(i)] > heap[i]) {

      // Swap child with parent
      int temp = heap[i];
      heap[i] = heap[parent(i)];
      heap[parent(i)] = temp;

      i = parent(i);  // Move to parent index
   }
}
```

```
// Heapify function to maintain Min Heap property
void heapify(int i) {

   int smallest = i;     // Assume current node is smallest
   int l = left(i);      // Get left child index
   int r = right(i);     // Get right child index

   // If left child exists and is smaller than current node
   if (l < size && heap[l] < heap[smallest])
      smallest = l;

   // If right child exists and is smaller than current smallest
   if (r < size && heap[r] < heap[smallest])
      smallest = r;

   // If smallest element is not the current node
   if (smallest != i) {

      // Swap current node with smallest child
      int temp = heap[i];
      heap[i] = heap[smallest];
      heap[smallest] = temp;

      // Recursively heapify the affected subtree
      heapify(smallest);
   }
}
```

```
// Function to delete root element (minimum element)
void deleteRoot() {

    // Check if heap is empty
    if (size <= 0) {
        System.out.println("Heap is empty!");
        return;
    }

    // Print deleted root element
    System.out.println("Deleted root element: " + heap[0]);

    // Replace root with last element
    heap[0] = heap[size - 1];

    size--;  // Reduce heap size

    // Restore heap property from root
    heapify(0);
}

// Function to display heap elements
void display() {

    System.out.print("Heap elements: ");

    // Print all heap elements
```

```
    for (int i = 0; i < size; i++)
        System.out.print(heap[i] + " ");

    System.out.println();
}

// Main method (program execution starts here)
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);  // Create Scanner object

    // Input heap capacity
    System.out.print("Enter heap capacity: ");
    int n = sc.nextInt();

    // Create Min Heap object
    MinHeapDeleteRoot h = new MinHeapDeleteRoot(n);

    // Input number of elements
    System.out.print("Enter number of elements: ");
    int m = sc.nextInt();

    System.out.println("Enter elements:");

    // Insert elements into heap
    for (int i = 0; i < m; i++)
        h.insert(sc.nextInt());
```

```
        System.out.println("Min Heap created successfully ✅ ");

        // Display heap before deletion
        h.display();

        // Delete root element
        h.deleteRoot();

        // Display heap after deletion
        h.display();
    }
}
```

2. Min Heap Delete Any Element

```
import java.util.Scanner;  // Import Scanner for user input

// Class for Min Heap operations
class MinHeapProgram {

    int heap[];     // Array to store heap elements
    int size;       // Current number of elements in heap
    int capacity;   // Maximum heap size

    // Constructor to initialize heap
    MinHeapProgram(int capacity) {
        this.capacity = capacity;    // Set heap capacity
```

```
    heap = new int[capacity];     // Create heap array
    size = 0;                     // Initially heap is empty
}

// Function to find parent index
int parent(int i) {
    return (i - 1) / 2;
}

// Function to find left child index
int left(int i) {
    return 2 * i + 1;
}

// Function to find right child index
int right(int i) {
    return 2 * i + 2;
}

// Function to insert element into heap
void insert(int key) {

    // Check heap overflow condition
    if (size == capacity) {
        System.out.println("Heap overflow!");
        return;
    }
```

```
    size++;             // Increase heap size
    int i = size - 1;     // Insert element at last position
    heap[i] = key;

    // Fix heap property (move element upward if needed)
    while (i != 0 && heap[parent(i)] > heap[i]) {

        // Swap child with parent
        int temp = heap[i];
        heap[i] = heap[parent(i)];
        heap[parent(i)] = temp;

        i = parent(i);    // Move upward
    }
}

// Heapify function to restore Min Heap property
void heapify(int i) {

    int smallest = i;    // Assume current node is smallest
    int l = left(i);     // Get left child index
    int r = right(i);    // Get right child index

    // Check if left child is smaller
    if (l < size && heap[l] < heap[smallest])
        smallest = l;

    // Check if right child is smaller
```

```
    if (r < size && heap[r] < heap[smallest])
        smallest = r;

    // If smallest is not current node
    if (smallest != i) {

        // Swap current node with smallest child
        int temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;

        // Recursively heapify affected subtree
        heapify(smallest);
    }
}

// Function to delete a specific element from heap
void deleteElement(int value) {

    int index = -1;   // Variable to store index of element

    // Search element inside heap
    for (int i = 0; i < size; i++) {
        if (heap[i] == value) {
            index = i;
            break;
        }
    }
```

```
    // If element not found
    if (index == -1) {
        System.out.println("Element not found in heap!");
        return;
    }

    // Replace element with last element
    heap[index] = heap[size - 1];

    size--;  // Reduce heap size

    // Restore heap property from changed index
    heapify(index);

    System.out.println("Element deleted successfully ✅ ");
}

// Function to display heap elements
void display() {

    System.out.print("Heap elements: ");

    // Print heap elements
    for (int i = 0; i < size; i++)
        System.out.print(heap[i] + " ");

    System.out.println();
```

```
}

// Main method (program execution starts here)
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);  // Create Scanner object

    // Input heap capacity
    System.out.print("Enter heap capacity: ");
    int n = sc.nextInt();

    // Create Min Heap object
    MinHeapProgram h = new MinHeapProgram(n);

    // Input number of elements
    System.out.print("Enter number of elements: ");
    int m = sc.nextInt();

    System.out.println("Enter elements:");

    // Insert elements into heap
    for (int i = 0; i < m; i++)
        h.insert(sc.nextInt());

    // Display heap before deletion
    h.display();

    // Input element to delete
```

```
        System.out.print("Enter element to delete: ");
        int del = sc.nextInt();

        // Delete specified element
        h.deleteElement(del);

        // Display heap after deletion
        h.display();
    }
}
```

BFS using Adjacency Matrix

```
import java.util.*;   // Import Scanner, Queue, LinkedList classes

// Class for BFS traversal using adjacency matrix
class BFSMatrix {

    int vertices;        // Number of vertices in graph
    int matrix[][];      // Adjacency matrix to store edges
    boolean visited[];    // Array to track visited vertices

    // Constructor to initialize graph
    BFSMatrix(int v) {
        vertices = v;                // Assign number of vertices
        matrix = new int[v][v];        // Create adjacency matrix
        visited = new boolean[v];      // Initially all vertices unvisited
```

```
}

// Function to add edge between two vertices
void addEdge(int src, int dest) {

    matrix[src][dest] = 1;        // Mark edge from src to dest
    matrix[dest][src] = 1;        // Mark edge from dest to src (undirected graph)

    // Remove above line if graph is directed
}

// Function to perform BFS traversal
void BFS(int start) {

    Queue<Integer> q = new LinkedList<>();   // Create queue for BFS

    visited[start] = true;   // Mark starting vertex as visited
    q.add(start);          // Insert starting vertex into queue

    System.out.print("BFS Traversal: ");

    // Continue until queue becomes empty
    while (!q.isEmpty()) {

        int node = q.poll();     // Remove element from queue
        System.out.print(node + " ");   // Print visited node

        // Check all adjacent vertices of current node
```

```
        for (int i = 0; i < vertices; i++) {

            // If edge exists and vertex not visited
            if (matrix[node][i] == 1 && !visited[i]) {

                visited[i] = true;   // Mark vertex as visited
                q.add(i);          // Add vertex into queue
            }
        }
    }
}

// Main method (execution starts here)
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);   // Create Scanner object

    // Input number of vertices
    System.out.print("Enter number of vertices: ");
    int v = sc.nextInt();

    BFSMatrix g = new BFSMatrix(v);   // Create graph object

    // Input number of edges
    System.out.print("Enter number of edges: ");
    int e = sc.nextInt();

    System.out.println("Enter edges (source destination):");
```

```
        // Input edges
        for (int i = 0; i < e; i++) {

            int src = sc.nextInt();
            int dest = sc.nextInt();

            g.addEdge(src, dest);   // Add edge to graph
        }

        // Input starting vertex for BFS
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();

        g.BFS(start);   // Call BFS traversal function
    }
}
```

BFS using Adjacency List

```
import java.util.*;   // Import Scanner, Queue, LinkedList classes

// Class for BFS traversal using Adjacency List
class BFSAdjList {

    int vertices;                // Number of vertices
    LinkedList<Integer> adjList[];   // Array of linked lists to store graph
```

```
// Constructor to initialize adjacency list
BFSAdjList(int v) {

   vertices = v;                  // Set number of vertices
   adjList = new LinkedList[v];      // Create array of linked lists

   // Initialize each linked list
   for (int i = 0; i < v; i++)
      adjList[i] = new LinkedList<>();
}

// Function to add edge between two vertices
void addEdge(int src, int dest) {

   adjList[src].add(dest);   // Add destination to source list
   adjList[dest].add(src);   // Add source to destination list (undirected graph)

   // Remove above line if graph is directed
}

// Function to perform BFS traversal
void BFS(int start) {

   boolean visited[] = new boolean[vertices];  // Track visited vertices

   Queue<Integer> queue = new LinkedList<>();  // Create queue for BFS

   visited[start] = true;   // Mark starting vertex visited
```

```
    queue.add(start);       // Insert starting vertex into queue

    System.out.print("BFS Traversal: ");

    // Continue until queue becomes empty
    while (!queue.isEmpty()) {

        int node = queue.poll();   // Remove element from queue
        System.out.print(node + " ");   // Print current node

        // Visit all neighbors of current node
        for (int neighbor : adjList[node]) {

            // If neighbor not visited
            if (!visited[neighbor]) {

                visited[neighbor] = true;   // Mark visited
                queue.add(neighbor);       // Add to queue
            }
        }
    }
}

// Main method (program execution starts here)
public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);   // Create Scanner object
```

```
        // Input number of vertices
        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        BFSAdjList g = new BFSAdjList(v);   // Create graph object

        // Input number of edges
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (source destination):");

        // Input edges one by one
        for (int i = 0; i < e; i++) {

            int src = sc.nextInt();
            int dest = sc.nextInt();

            g.addEdge(src, dest);   // Add edge into graph
        }

        // Input starting vertex
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();

        g.BFS(start);   // Call BFS function
    }
}
```

DFS using Adjacency Matrix

```
import java.util.*;   // Import Scanner class for input

// Class for DFS traversal using adjacency matrix
class DFSMatrix {

  int vertices;        // Number of vertices
  int matrix[][];      // Adjacency matrix to store edges
  boolean visited[];    // Array to track visited nodes

  // Constructor to initialize graph
  DFSMatrix(int v) {

    vertices = v;              // Assign number of vertices
    matrix = new int[v][v];      // Create adjacency matrix
    visited = new boolean[v];    // Initially all nodes unvisited
  }

  // Function to add edge between two vertices
  void addEdge(int src, int dest) {

    matrix[src][dest] = 1;   // Edge from source to destination
    matrix[dest][src] = 1;   // Edge from destination to source (undirected graph)

    // Remove above line if graph is directed
  }
```

```
// Recursive DFS function
void DFS(int node) {

   visited[node] = true;        // Mark current node as visited
   System.out.print(node + " ");  // Print current node

   // Visit all adjacent vertices
   for (int i = 0; i < vertices; i++) {

      // If edge exists and node not visited
      if (matrix[node][i] == 1 && !visited[i]) {

         DFS(i);   // Recursive call
      }
   }
}

// Main method (execution starts here)
public static void main(String args[]) {

   Scanner sc = new Scanner(System.in);   // Create Scanner object

   // Input number of vertices
   System.out.print("Enter number of vertices: ");
   int v = sc.nextInt();

   DFSMatrix g = new DFSMatrix(v);   // Create graph object
```

```
        // Input number of edges
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (source destination):");

        // Input edges
        for (int i = 0; i < e; i++) {

            int src = sc.nextInt();
            int dest = sc.nextInt();

            g.addEdge(src, dest);   // Add edge into matrix
        }

        // Input starting vertex
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();

        System.out.print("DFS Traversal: ");

        g.DFS(start);   // Call DFS function
    }
}
```

DFS Using Adjacency List Representation

```
import java.util.*;   // Import Scanner and LinkedList classes

// Class for DFS traversal using adjacency list
class DFSAdjList {

  int vertices;                // Number of vertices
  LinkedList<Integer> adjList[];   // Array of linked lists

  // Constructor to initialize adjacency list
  DFSAdjList(int v) {

    vertices = v;                // Assign number of vertices
    adjList = new LinkedList[v];     // Create array of linked lists

    // Initialize each linked list
    for (int i = 0; i < v; i++)
      adjList[i] = new LinkedList<>();
  }

  // Function to add edge between two vertices
  void addEdge(int src, int dest) {

    adjList[src].add(dest);   // Add destination to source list
    adjList[dest].add(src);   // Add source to destination list (undirected graph)

    // Remove above line if graph is directed
```

```
}

// Recursive DFS function
void DFS(int node, boolean visited[]) {

   visited[node] = true;        // Mark current node as visited
   System.out.print(node + " ");  // Print current node

   // Visit all neighbors of current node
   for (int neighbor : adjList[node]) {

      // If neighbor not visited
      if (!visited[neighbor]) {

         DFS(neighbor, visited);   // Recursive call
      }
   }
}

// Main method (program execution starts here)
public static void main(String args[]) {

   Scanner sc = new Scanner(System.in);   // Create Scanner object

   // Input number of vertices
   System.out.print("Enter number of vertices: ");
   int v = sc.nextInt();
```

```
        DFSAdjList g = new DFSAdjList(v);   // Create graph object

        // Input number of edges
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (source destination):");

        // Input edges one by one
        for (int i = 0; i < e; i++) {

            int src = sc.nextInt();
            int dest = sc.nextInt();

            g.addEdge(src, dest);   // Add edge into adjacency list
        }

        boolean visited[] = new boolean[v];   // Array to track visited nodes

        // Input starting vertex
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();

        System.out.print("DFS Traversal: ");

        g.DFS(start, visited);   // Call DFS function
    }
}
```

9. Biconnected Component

```
import java.util.*;

// Class to find Biconnected Components using DFS
class BiconnectedComponents {

  // Edge structure to store graph edges
  static class Edge {

    int u, v;

    Edge(int u, int v) {
      this.u = u;
      this.v = v;
    }
  }

  int V;                    // Number of vertices
  LinkedList<Integer>[] adj;    // Adjacency list
  int time = 0;             // Timer for DFS discovery time

  // Constructor
  BiconnectedComponents(int v) {

    V = v;
    adj = new LinkedList[v];
```

```
    // Initialize adjacency list
    for (int i = 0; i < v; i++)
        adj[i] = new LinkedList<>();
}

// Function to add edge in graph
void addEdge(int u, int v) {

    adj[u].add(v);
    adj[v].add(u); // Undirected graph
}

// DFS utility function to find BCC
void BCCUtil(int u, int parent,
            int[] disc,
            int[] low,
            Stack<Edge> st) {

    // Initialize discovery time and low value
    disc[u] = low[u] = ++time;

    int children = 0; // Count children in DFS tree

    // Traverse all adjacent vertices
    for (int v : adj[u]) {

        // If v not visited
        if (disc[v] == -1) {
```

```
children++;

// Push edge into stack
st.push(new Edge(u, v));

// Recursive DFS call
BCCUtil(v, u, disc, low, st);

// Update low value of u
low[u] = Math.min(low[u], low[v]);

// Check articulation condition
if ((parent == -1 && children > 1) ||
   (parent != -1 && low[v] >= disc[u])) {

   System.out.println("Biconnected Component:");

   Edge e;

   // Pop edges until (u,v) edge found
   do {

      e = st.pop();
      System.out.println(e.u + " - " + e.v);

   } while (!(e.u == u && e.v == v));
```

```
            System.out.println();
        }
    }

    // Back edge case
    else if (v != parent && disc[v] < disc[u]) {

        // Update low value
        low[u] = Math.min(low[u], disc[v]);

        // Push edge into stack
        st.push(new Edge(u, v));
    }
  }
}

// Function to find all Biconnected Components
void findBCC() {

  int[] disc = new int[V]; // Discovery time array
  int[] low = new int[V];  // Low value array

  Arrays.fill(disc, -1);   // Mark all nodes unvisited

  Stack<Edge> st = new Stack<>(); // Stack to store edges

  // Call DFS for each unvisited vertex
  for (int i = 0; i < V; i++) {
```

```
        if (disc[i] == -1) {

            BCCUtil(i, -1, disc, low, st);

            // Print remaining edges in stack
            if (!st.isEmpty()) {

                System.out.println("Biconnected Component:");

                while (!st.isEmpty()) {

                    Edge e = st.pop();
                    System.out.println(e.u + " - " + e.v);
                }

                System.out.println();
            }
        }
    }
}

// Main method
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Input number of vertices
```

```
        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        // Create graph object
        BiconnectedComponents g =
            new BiconnectedComponents(v);

        // Input number of edges
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (u v):");

        // Input edges
        for (int i = 0; i < e; i++) {

            int u = sc.nextInt();
            int w = sc.nextInt();

            g.addEdge(u, w);
        }

        // Find Biconnected Components
        System.out.println("\nBiconnected Components are:\n");

        g.findBCC();
    }
}
```

Quick Sort

```
import java.util.*;   // Import Scanner class

// Class for Quick Sort with execution time calculation
class QuickSortTime {

  // Function to perform Quick Sort
  static void quickSort(int arr[], int low, int high) {

    // Execute only if valid range exists
    if (low < high) {

      // Partition array and get pivot index
      int pi = partition(arr, low, high);

      // Sort left subarray
      quickSort(arr, low, pi - 1);

      // Sort right subarray
      quickSort(arr, pi + 1, high);
    }
  }

  // Partition function
  static int partition(int arr[], int low, int high) {

    int pivot = arr[high];   // Choose last element as pivot
```

```
    int i = low - 1;        // Pointer for smaller element

    // Traverse array from low to high-1
    for (int j = low; j < high; j++) {

        // If current element smaller than pivot
        if (arr[j] < pivot) {

            i++;  // Move index forward

            // Swap arr[i] and arr[j]
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // Place pivot at correct position
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;   // Return pivot index
}

// Function to print array
static void printArray(int arr[]) {
```

```
    for (int num : arr)
        System.out.print(num + " ");
}

// Main method (execution starts here)
public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    // Input number of elements
    System.out.print("Enter number of elements: ");
    int n = sc.nextInt();

    int arr[] = new int[n];

    // Input array elements
    System.out.println("Enter elements:");

    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();

    // Start execution time measurement
    long startTime = System.nanoTime();

    // Call Quick Sort function
    quickSort(arr, 0, n - 1);
```

```
        // End execution time measurement
        long endTime = System.nanoTime();

        // Print sorted array
        System.out.println("Sorted array:");
        printArray(arr);

        // Calculate execution time
        long executionTime = endTime - startTime;

        // Display execution time
        System.out.println("\nExecution Time: "
                + executionTime + " nanoseconds");
    }
}
```

Merge Sort

```
import java.util.*;   // Import Scanner class

// Class for Merge Sort with execution time calculation
class MergeSortTime {

    // Function to merge two sorted subarrays
    static void merge(int arr[], int left, int mid, int right) {

        // Size of left and right subarrays
```

```
int n1 = mid - left + 1;
int n2 = right - mid;

// Create temporary arrays
int L[] = new int[n1];
int R[] = new int[n2];

// Copy data into left array
for (int i = 0; i < n1; i++)
   L[i] = arr[left + i];

// Copy data into right array
for (int j = 0; j < n2; j++)
   R[j] = arr[mid + 1 + j];

int i = 0, j = 0;

int k = left;  // Initial index of merged array

// Merge temporary arrays into original array
while (i < n1 && j < n2) {

   // Compare elements of both arrays
   if (L[i] <= R[j]) {

      arr[k] = L[i];
      i++;
```

```
        } else {

            arr[k] = R[j];
            j++;
        }

        k++;
    }

    // Copy remaining elements of left array (if any)
    while (i < n1) {

        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of right array (if any)
    while (j < n2) {

        arr[k] = R[j];
        j++;
        k++;
    }
}

// Recursive function for Merge Sort
static void mergeSort(int arr[], int left, int right) {
```

```
    // Continue dividing until one element remains
    if (left < right) {

        // Find middle index
        int mid = (left + right) / 2;

        // Sort first half
        mergeSort(arr, left, mid);

        // Sort second half
        mergeSort(arr, mid + 1, right);

        // Merge both halves
        merge(arr, left, mid, right);
    }
}

// Function to print array
static void printArray(int arr[]) {

    for (int x : arr)
        System.out.print(x + " ");
}

// Main method (program execution starts here)
public static void main(String args[]) {
```

```
Scanner sc = new Scanner(System.in);

// Input number of elements
System.out.print("Enter number of elements: ");
int n = sc.nextInt();

int arr[] = new int[n];

// Input array elements
System.out.println("Enter elements:");

for (int i = 0; i < n; i++)
    arr[i] = sc.nextInt();

// Start time measurement
long startTime = System.nanoTime();

// Call Merge Sort
mergeSort(arr, 0, n - 1);

// End time measurement
long endTime = System.nanoTime();

// Print sorted array
System.out.println("Sorted array:");
printArray(arr);

// Calculate execution time
```

```
        long executionTime = endTime - startTime;

        // Display execution time
        System.out.println("\nExecution Time: "
                + executionTime + " nanoseconds");
    }
}
```

Fractional Knapsack

```
import java.util.*;   // Import Scanner and Arrays classes

// Class for Fractional Knapsack Problem
class FractionalKnapsack {

    // Item class to store weight, profit, and ratio
    static class Item {

        int weight, profit;
        double ratio;

        // Constructor to initialize item values
        Item(int w, int p) {

            weight = w;
            profit = p;
```

```
        // Calculate profit-to-weight ratio
        ratio = (double) p / w;
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Input number of items
    System.out.print("Enter number of items: ");
    int n = sc.nextInt();

    Item[] items = new Item[n];   // Create array of items

    // Input weight and profit of each item
    System.out.println("Enter weight and profit of each item:");

    for (int i = 0; i < n; i++) {

        int w = sc.nextInt();
        int p = sc.nextInt();

        items[i] = new Item(w, p);  // Create item object
    }

    // Input knapsack capacity
    System.out.print("Enter knapsack capacity: ");
```

```
int capacity = sc.nextInt();

// Sort items based on descending ratio
Arrays.sort(items,
      (a, b) -> Double.compare(b.ratio, a.ratio));

double totalProfit = 0.0;   // Variable to store maximum profit

// Select items greedily
for (int i = 0; i < n; i++) {

   // If item fits completely
   if (capacity >= items[i].weight) {

      capacity -= items[i].weight;   // Reduce capacity

      totalProfit += items[i].profit;  // Add full profit

      System.out.println("Selected item fully (W="
            + items[i].weight + ", P="
            + items[i].profit + ")");

   }

   // If only fraction can be taken
   else {

      double fraction =
```

```
                (double) capacity / items[i].weight;

            totalProfit += items[i].profit * fraction;

            System.out.println("Selected "
                + fraction + " fraction of item (W="
                + items[i].weight + ", P="
                + items[i].profit + ")");

            break;  // Knapsack becomes full
        }
    }

    // Print maximum profit
    System.out.println("Maximum Profit = " + totalProfit);
  }
}
```

Job Sequencing

```
import java.util.*;   // Import Scanner and Arrays classes

// Class for Job Sequencing with Deadlines
class JobSequencing {

  // Job class to store job id, deadline and profit
  static class Job {
```

```
    String id;   // Job name
    int deadline; // Deadline of job
    int profit;   // Profit of job

    // Constructor
    Job(String i, int d, int p) {

        id = i;
        deadline = d;
        profit = p;
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Input number of jobs
    System.out.print("Enter number of jobs: ");
    int n = sc.nextInt();

    Job jobs[] = new Job[n];  // Create array of jobs

    // Input job details
    System.out.println("Enter job id, deadline and profit:");

    for (int i = 0; i < n; i++) {
```

```
    String id = sc.next();
    int deadline = sc.nextInt();
    int profit = sc.nextInt();

    jobs[i] = new Job(id, deadline, profit);
}

// Step 1: Sort jobs by descending order of profit
Arrays.sort(jobs, (a, b) -> b.profit - a.profit);

// Step 2: Find maximum deadline
int maxDeadline = 0;

for (Job job : jobs)
    maxDeadline = Math.max(maxDeadline, job.deadline);

// Array to store job sequence
String result[] = new String[maxDeadline];

// Array to track free time slots
boolean slot[] = new boolean[maxDeadline];

int totalProfit = 0;

// Step 3: Assign jobs to slots
for (int i = 0; i < n; i++) {
```

```
    // Find free slot from deadline backwards
    for (int j = Math.min(maxDeadline, jobs[i].deadline) - 1;
        j >= 0;
        j--) {

        // If slot available
        if (!slot[j]) {

            slot[j] = true;             // Mark slot occupied
            result[j] = jobs[i].id;      // Store job id
            totalProfit += jobs[i].profit; // Add profit

            break;
        }
    }
}

// Step 4: Print selected job sequence
System.out.print("Selected job sequence: ");

for (int i = 0; i < maxDeadline; i++) {

    if (slot[i])
        System.out.print(result[i] + " ");
}

// Print maximum profit
System.out.println("\nMaximum Profit: " + totalProfit);
```

```
    }
}
```

Dijkshtas

```
import java.util.*;

// Class for Dijkstra's Algorithm
class Dijkstra {

    static final int INF = 9999; // Represent infinity

    // Function to find vertex with minimum distance
    static int minDistance(int dist[], boolean visited[], int V) {

        int min = INF, minIndex = -1;

        for (int i = 0; i < V; i++) {

            if (!visited[i] && dist[i] < min) {

                min = dist[i];
                minIndex = i;
            }
        }

        return minIndex;
    }
```

```
// Function to implement Dijkstra's Algorithm
static void dijkstra(int graph[][], int src, int V) {

    int dist[] = new int[V];      // Distance array
    boolean visited[] = new boolean[V]; // Track visited vertices

    // Initialize distances
    for (int i = 0; i < V; i++) {

        dist[i] = INF;
        visited[i] = false;
    }

    dist[src] = 0; // Distance from source to itself is 0

    // Find shortest path for all vertices
    for (int count = 0; count < V - 1; count++) {

        int u = minDistance(dist, visited, V);

        visited[u] = true;

        // Update distances of adjacent vertices
        for (int v = 0; v < V; v++) {

            if (!visited[v] &&
                graph[u][v] != 0 &&
```

```
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
        }
    }
  }

  // Print shortest distances
  System.out.println("Vertex\tDistance from Source");

  for (int i = 0; i < V; i++)
    System.out.println(i + "\t" + dist[i]);
}

// Main method
public static void main(String args[]) {

  Scanner sc = new Scanner(System.in);

  System.out.print("Enter number of vertices: ");
  int V = sc.nextInt();

  int graph[][] = new int[V][V];

  System.out.println("Enter adjacency matrix:");

  for (int i = 0; i < V; i++)
```

```
        for (int j = 0; j < V; j++)
            graph[i][j] = sc.nextInt();

    System.out.print("Enter source vertex: ");
    int src = sc.nextInt();

    dijkstra(graph, src, V);
  }
}
```

0/1 knapsack dp

```
import java.util.*;

// Class for 0/1 Knapsack using Dynamic Programming
class KnapsackDP {

  // Function to solve knapsack problem
  static int knapsack(int W, int wt[], int val[], int n) {

    // Create DP table
    int dp[][] = new int[n + 1][W + 1];

    // Build table row by row
    for (int i = 0; i <= n; i++) {

        for (int w = 0; w <= W; w++) {
```

```
        // Base case: no items or zero capacity
        if (i == 0 || w == 0)
          dp[i][w] = 0;

        // If item weight is less than capacity
        else if (wt[i - 1] <= w)

          dp[i][w] = Math.max(
                val[i - 1] +
                dp[i - 1][w - wt[i - 1]],
                dp[i - 1][w]);

        // If item cannot be included
        else
          dp[i][w] = dp[i - 1][w];
      }
    }

    // Return maximum profit
    return dp[n][W];
  }

  public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    // Input number of items
    System.out.print("Enter number of items: ");
```

```
        int n = sc.nextInt();

        int wt[] = new int[n];
        int val[] = new int[n];

        // Input weights
        System.out.println("Enter weights:");
        for (int i = 0; i < n; i++)
            wt[i] = sc.nextInt();

        // Input profits
        System.out.println("Enter profits:");
        for (int i = 0; i < n; i++)
            val[i] = sc.nextInt();

        // Input knapsack capacity
        System.out.print("Enter capacity: ");
        int W = sc.nextInt();

        // Call knapsack function
        int maxProfit = knapsack(W, wt, val, n);

        // Display result
        System.out.println("Maximum Profit = " + maxProfit);
    }
}
```

Edit distance

```
import java.util.*;

// Class to calculate minimum edit distance
class EditDistance {

  // Function to compute edit distance
  static int editDistance(String s1, String s2) {

    int m = s1.length();
    int n = s2.length();

    // Create DP table
    int dp[][] = new int[m + 1][n + 1];

    // Fill DP table
    for (int i = 0; i <= m; i++) {

      for (int j = 0; j <= n; j++) {

        // If first string is empty
        if (i == 0)
          dp[i][j] = j;

        // If second string is empty
        else if (j == 0)
          dp[i][j] = i;
```

```
        // If characters match
        else if (s1.charAt(i - 1) == s2.charAt(j - 1))
          dp[i][j] = dp[i - 1][j - 1];

        // If characters differ
        else
          dp[i][j] = 1 + Math.min(
                dp[i - 1][j],       // delete
                Math.min(
                  dp[i][j - 1],    // insert
                  dp[i - 1][j - 1] // replace
                )
          );
      }
    }

    // Return final answer
    return dp[m][n];
  }

  public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    // Input first string
    System.out.print("Enter first string: ");
    String s1 = sc.nextLine();
```

```
        // Input second string
        System.out.print("Enter second string: ");
        String s2 = sc.nextLine();

        // Compute edit distance
        int result = editDistance(s1, s2);

        // Print result
        System.out.println("Minimum Edit Distance = " + result);
    }
}
```

N QUEENS

```
import java.util.*;

// Class to solve N-Queens problem using Backtracking
class NQueens {

    int N;          // Size of chessboard
    int board[][];     // Chessboard

    // Constructor
    NQueens(int n) {

        N = n;
        board = new int[N][N];
    }
```

```
// Function to check if queen can be placed safely
boolean isSafe(int row, int col) {

  // Check left side of same row
  for (int i = 0; i < col; i++)
    if (board[row][i] == 1)
      return false;

  // Check upper diagonal
  for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
    if (board[i][j] == 1)
      return false;

  // Check lower diagonal
  for (int i = row, j = col; i < N && j >= 0; i++, j--)
    if (board[i][j] == 1)
      return false;

  return true;
}

// Recursive function to solve N-Queens
boolean solveNQueens(int col) {

  // If all queens placed successfully
  if (col >= N)
    return true;
```

```
    // Try placing queen in each row
    for (int i = 0; i < N; i++) {

        // Check if safe
        if (isSafe(i, col)) {

            board[i][col] = 1; // Place queen

            // Recur for next column
            if (solveNQueens(col + 1))
                return true;

            // Backtrack if placement fails
            board[i][col] = 0;
        }
    }

    return false;
}

// Function to print solution
void printSolution() {

    for (int i = 0; i < N; i++) {

        for (int j = 0; j < N; j++)
            System.out.print(board[i][j] + " ");
```

```
            System.out.println();
        }
    }

    // Main method
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        // Input number of queens
        System.out.print("Enter number of queens: ");
        int n = sc.nextInt();

        NQueens obj = new NQueens(n);

        // Solve problem
        if (obj.solveNQueens(0)) {

            System.out.println("Solution exists:");
            obj.printSolution();

        } else {

            System.out.println("No solution exists.");
        }
    }
}
```

GRAPH COLORING

```
import java.util.*;

// Class to solve Graph Coloring using Backtracking
class GraphColoring {

  int V;          // Number of vertices
  int graph[][];    // Adjacency matrix
  int m;          // Number of colors
  int color[];      // Color array

  // Constructor
  GraphColoring(int V, int m) {

    this.V = V;
    this.m = m;

    graph = new int[V][V];
    color = new int[V]; // Initially all colors = 0
  }

  // Function to check whether it is safe to assign color c to vertex v
  boolean isSafe(int v, int c) {

    for (int i = 0; i < V; i++) {
```

```
        // If adjacent vertex has same color
        if (graph[v][i] == 1 && color[i] == c)
            return false;
    }

    return true;
}

// Recursive function to solve coloring problem
boolean solve(int v) {

    // If all vertices are colored
    if (v == V)
        return true;

    // Try all colors from 1 to m
    for (int c = 1; c <= m; c++) {

        // Check if safe
        if (isSafe(v, c)) {

            color[v] = c; // Assign color

            // Recur for next vertex
            if (solve(v + 1))
                return true;

            // Backtrack if not possible
```

```
            color[v] = 0;
        }
    }

    return false;
}

// Function to print solution
void printSolution() {

    System.out.println("Color Assignment:");

    for (int i = 0; i < V; i++)
        System.out.println("Vertex " + i +
                           " -> Color " + color[i]);
}

// Main method
public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    // Input number of vertices
    System.out.print("Enter number of vertices: ");
    int V = sc.nextInt();

    // Input number of colors
    System.out.print("Enter number of colors: ");
```

```
        int m = sc.nextInt();

        GraphColoring g = new GraphColoring(V, m);

        // Input adjacency matrix
        System.out.println("Enter adjacency matrix:");

        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                g.graph[i][j] = sc.nextInt();

        // Solve graph coloring
        if (g.solve(0)) {

            System.out.println("Solution exists:");
            g.printSolution();

        } else {

            System.out.println("No solution exists.");
        }
    }
}
```

subset

```
import java.util.*;

// Class to solve Sum of Subsets using Backtracking
class SumOfSubsets {

  int set[];      // Input set
  int subset[];    // Array to store current subset
  int n;          // Number of elements
  int target;      // Target sum

  // Constructor
  SumOfSubsets(int set[], int n, int target) {

    this.set = set;
    this.n = n;
    this.target = target;

    subset = new int[n];
  }

  // Recursive function for Sum of Subsets
  void findSubset(int index, int currentSum, int subsetSize) {

    // If current sum equals target, print solution
    if (currentSum == target) {
```

```
        System.out.print("Subset found: ");

        for (int i = 0; i < subsetSize; i++)
            System.out.print(subset[i] + " ");

        System.out.println();

        return;
    }

    // If sum exceeds target or no elements left
    if (index == n || currentSum > target)
        return;

    // Include current element
    subset[subsetSize] = set[index];

    findSubset(index + 1,
            currentSum + set[index],
            subsetSize + 1);

    // Exclude current element (backtracking step)
    findSubset(index + 1,
            currentSum,
            subsetSize);
}
```

```
    // Main method
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        // Input number of elements
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int set[] = new int[n];

        // Input set elements
        System.out.println("Enter elements:");

        for (int i = 0; i < n; i++)
            set[i] = sc.nextInt();

        // Input target sum
        System.out.print("Enter target sum: ");
        int target = sc.nextInt();

        SumOfSubsets obj =
                new SumOfSubsets(set, n, target);

        // Call function
        obj.findSubset(0, 0, 0);
    }
}
```

TSP USING BRANCH AND BOUND

```
import java.util.*;

class TSPBranchBound {

    static int N;              // Number of cities
    static int final_res = Integer.MAX_VALUE; // Final minimum cost
    static int final_path[];     // Stores final solution path

    // Function to copy temporary solution to final solution
    static void copyToFinal(int curr_path[]) {

        for (int i = 0; i < N; i++)
            final_path[i] = curr_path[i];

        final_path[N] = curr_path[0];
    }

    // Function to find minimum edge cost
    static int firstMin(int adj[][], int i) {

        int min = Integer.MAX_VALUE;

        for (int k = 0; k < N; k++)
            if (adj[i][k] < min && i != k)
                min = adj[i][k];
```

```
    return min;
}

// Function to find second minimum edge cost
static int secondMin(int adj[][], int i) {

    int first = Integer.MAX_VALUE, second = Integer.MAX_VALUE;

    for (int j = 0; j < N; j++) {

        if (i == j)
            continue;

        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        }

        else if (adj[i][j] < second &&
                adj[i][j] != first)
            second = adj[i][j];
    }

    return second;
}

// Recursive function for Branch and Bound
```

```
static void TSPRec(int adj[][],
                   int curr_bound,
                   int curr_weight,
                   int level,
                   int curr_path[],
                   boolean visited[]) {

  // If all cities visited
  if (level == N) {

    if (adj[curr_path[level - 1]][curr_path[0]] != 0) {

      int curr_res =
          curr_weight +
          adj[curr_path[level - 1]][curr_path[0]];

      if (curr_res < final_res) {

        copyToFinal(curr_path);
        final_res = curr_res;
      }
    }
    return;
  }

  // Try all possible next cities
  for (int i = 0; i < N; i++) {
```

```
if (adj[curr_path[level - 1]][i] != 0 &&
    !visited[i]) {

    int temp = curr_bound;

    curr_weight +=
          adj[curr_path[level - 1]][i];

    // Calculate new bound
    if (level == 1)
      curr_bound -=
            ((firstMin(adj,
              curr_path[level - 1]) +
              firstMin(adj, i)) / 2);

    else
      curr_bound -=
            ((secondMin(adj,
              curr_path[level - 1]) +
              firstMin(adj, i)) / 2);

    // If promising path
    if (curr_bound + curr_weight < final_res) {

      curr_path[level] = i;
      visited[i] = true;

      TSPRec(adj,
```

```
                curr_bound,
                curr_weight,
                level + 1,
                curr_path,
                visited);
        }

        // Backtrack
        curr_weight -=
              adj[curr_path[level - 1]][i];

        curr_bound = temp;

        Arrays.fill(visited, false);

        for (int j = 0; j < level; j++)
           visited[curr_path[j]] = true;
      }
   }
}

// Main function to solve TSP
static void TSP(int adj[][]) {

   int curr_path[] = new int[N + 1];
   boolean visited[] = new boolean[N];

   int curr_bound = 0;
```

```
    Arrays.fill(curr_path, -1);

    // Initial bound calculation
    for (int i = 0; i < N; i++)
        curr_bound +=
                (firstMin(adj, i) +
                 secondMin(adj, i));

    curr_bound = (curr_bound % 2 == 1)
            ? curr_bound / 2 + 1
            : curr_bound / 2;

    visited[0] = true;
    curr_path[0] = 0;

    TSPRec(adj,
            curr_bound,
            0,
            1,
            curr_path,
            visited);
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter number of cities: ");
        N = sc.nextInt();

        int adj[][] = new int[N][N];

        System.out.println("Enter cost matrix:");

        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                adj[i][j] = sc.nextInt();

        final_path = new int[N + 1];

        TSP(adj);

        System.out.println("Minimum cost: " + final_res);

        System.out.print("Path Taken: ");

        for (int i = 0; i <= N; i++)
            System.out.print(final_path[i] + " ");
    }
}
```